



Métrologie depuis un flux vidéo avec Python.

LAURENT Quentin, DRAUS Arthur
Projet ENSISA de CPB1
Année universitaire
2025/2026

Table des matières

1	Fiche de présentation du projet CPB1	2
1.1	Étudiants	2
1.2	Le rapport de projet	2
1.3	Formulaire d'informations sur le plagiat	3
2	Introduction	4
2.1	Description du Projet	4
2.2	Applications	4
2.3	Montage expérimental	4
2.4	Objectifs et contraintes	4
3	Code du projet	5
3.1	Organisation des fichiers	5
3.2	Main.py	5
3.3	Traitement.py	7
3.4	Mesures.py	7
3.4.1	Calcul du périmètre de l'objet.	7
3.4.2	Calcul de l'aire de l'objet.	8
3.4.3	Calcul des dimensions de l'objet.	8
3.4.4	Fonction mesure(contours).	9
3.5	Optique.py	9

1 Fiche de présentation du projet CPB1

1.1 Étudiants

LAURENT Quentin, DRAUS Arthur
Année universitaire 2025/2026.

1.2 Le rapport de projet

Titre du projet : "Métrologie depuis un flux vidéo avec Python."

Mots clés pour décrire le projet :
Python, Mesures, Vidéo

Court résumé du projet :
Court résumé du projet en anglais :

1.3 Formulaire d'informations sur le plagiat

Dans le règlement des examens validé par la CFVU de l'UHA, le plagiat est assimilé à une fraude. Le plagiat consiste à reproduire un texte, une partie d'un texte, des données ou des images, toute production (texte ou image), ou à paraphraser un texte sans indiquer la provenance ou l'auteur. Le plagiat enfreint les règles de la déontologie universitaire et il constitue une fraude. Le plagiat constitue également une atteinte au droit d'auteur et à la propriété intellectuelle, susceptible d'être assimilé à un délit de contrefaçon. En cas de plagiat dans un devoir, dossier, mémoire ou thèse, l'étudiant sera présenté à la section disciplinaire de l'université qui pourra prononcer des sanctions allant de l'avertissement à l'exclusion. Dans le cas où le plagiat est aussi caractérisé comme étant une contrefaçon, d'éventuelles poursuites judiciaires pourront s'ajouter à la procédure disciplinaire.

Je soussigné : LAURENT Quentin, DRAUS Arthur

Etudiants à l'Université de Haute Alsace en : 2025-2026

Niveau d'études : BAC

Formation ou parcours : CPB1 ENSISA

Reconnaissons avoir pris connaissance du formulaire d'information sur le plagiat.

Fait à Mulhouse 68200

Le 12/05/2026

Signatures :

2 Introduction

2.1 Description du Projet

Ce projet vise à mettre en place un système de métrologie en temps réel grâce à un flux vidéo acquis et traité grâce à python. Nous cherchons donc à pouvoir mesurer des objets placés devant une caméra grâce à y script Python.

2.2 Applications

- Ce système peut, par exemple permettre de détecter des objets sur une chaîne de production et de contrôler leur taille afin de mettre en évidence les produits défectueux, sans ralentir la production.
- Ce système peut également servir à créer des plans de coupe à l'échelle en mesurant la surface qu'un objet occupe sur l'image.
- Nous pouvons également trouver des applications dans la recherche. Nous pouvons prendre comme exemple l'archéologie où les chercheurs classifient souvent les objets découverts par taille. Ce travail de mesure peut être grandement simplifié par des outils de métrologie numériques similaires à notre projet.

2.3 Montage expérimental

- On utilise un appareil photo Canon EOS R50 avec un objectif à focale fixe de 50 mm afin de minimiser les aberrations optiques.
- On positionne l'appareil sur un trépied réglé à l'aide d'un niveau à bulle de hauteur connue.
- Le capteur doit être parallèle à la surface sur laquelle l'objet est placé.
- On utilise une grille pour faire ressortir l'objet mesuré de l'arrière plan.
- On connecte la caméra en USB à un ordinateur.
- Depuis l'ordinateur, on commence l'acquisition.

2.4 Objectifs et contraintes

À travers ce projet nous voulons récupérer les mesures d'un objet quelconque suivantes :

- Le périmètre de l'objet.
- La surface de la face de l'objet montrée à la caméra.
- Les dimensions de l'objet (longueur et largeur)

Afin de pouvoir mesurer correctement un objet, nous devons définir plusieurs contraintes :

- La calibration de l'appareil permet de convertir des mesures en pixels en millimètres.
- Nous devons définir la précision des mesures pour savoir quels objets nous pouvons mesurer correctement.
- Nous devons faire des algorithmes de traitement et de calibration d'image.

3 Code du projet

3.1 Organisation des fichiers

Le code de notre projet se compose de quatre fichiers :

- Le fichier "main.py" initie et structure notre code. C'est à partir de ce fichier que nous faisons nos mesures.
- Le fichier "Traitement.py" contient une fonction traitement prenant en paramètre l'image brut de la caméra et renvoie un tableau numpy contenant les différents contours de l'objet.
- Le fichier "Mesures.py" contient un ensemble de fonctions qui renvoient les différentes mesures en pixels du plus grand contour que nous trouvons.
- Le fichier "Optique.py" contient des fonctions qui renvoient le rapport de pixels par millimètre de l'image nous permettant de convertir nos mesures renvoyées par les fonctions du fichier "Mesures.py" en millimètres.

Dans l'ensemble de nos fichiers, nous allons utiliser les librairies OpenCV (cv2) pour la gestion de la caméra et numpy (np) pour la gestion de tableaux (images)

3.2 Main.py

Ce fichier contient la boucle principale de notre code à partir de laquelle nous appelons nos fonctions.

- On importe la bibliothèque OpenCV (cv2) ainsi que les trois autres fichiers du projet

```
1 import cv2
2 import Traitement as t
3 import mesures as m
4 import optique as o
```

- On définit les variables de calibration et de mesure ainsi qu'un compteur qui enregistre le nombre de mesures.
- On définit la variable cap qui va récupérer le flux vidéo de la caméra.

```
1 ratio_px_mm = -1
2 ratio_mm_px = -1
3 i=0
4 perimetre,aire,largeur,hauteur = (0,0,0,0)
5 cap= cv2.VideoCapture(0)
```

- On initie une boucle infinie dans laquelle on exécute le reste du programme.
- On définit une variable key qui enregistre la touche du clavier pressée.
- On définit la variable frame (type numpy.ndarray) qui stocke l'information du flux vidéo dans un tableau de pixels RVB.
- On définit une variable ret (type booléen) qui renvoie True si cap enregistre un flux vidéo et False sinon.

```
1 while True:
2
3     key = cv2.waitKey(1)
4     ret,frame=cap.read()
5
```

```

6         if not ret: break
7         contours = t.traitement(frame)

1
2
3         if ratio_px_mm == -1:
4
5             cv2.putText(frame, "Appuyer sur 'c' pour calibrer la camera", (40,
6                 225), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 1)
7
8             if key == ord("c"):
9
10                 ratio_px_mm, ratio_mm_px = o.calibration(frame)

```

- On vérifie si le ratio de calibration n'a pas encore été défini (valeur par défaut -1).
- Si c'est le cas, on affiche un message à l'écran invitant l'utilisateur à appuyer sur 'c' pour calibrer la caméra.
- Si la touche 'c' est pressée, on appelle la fonction de calibration du fichier optique qui retourne les deux ratios de conversion pixels/mm et mm/pixels.

```

1         else :
2             if key & 0xFF == ord(" "):
3
4                 i+=1
5                 perimetre, aire, largeur, hauteur = m.mesures(contours)
6                 print(f"Mesure {i} : \nPerimetre : {perimetre*ratio_mm_px:.2f} mm\nAire : {aire*ratio_mm_px**2:.2f} mm2\nDimensions : {largeur*ratio_mm_px:.2f}x{hauteur*ratio_mm_px:.2f} mm")
7                 cv2.putText(frame, "Appuyer sur espace pour prendre une mesure", (40, 200), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 1)

```

- Une fois la calibration effectuée, on entre dans le bloc else.
- Si l'utilisateur appuie sur la touche espace, on incrémente le compteur de mesures i, puis on appelle la fonction mesures du fichier mesures qui retourne le périmètre, l'aire, la largeur et la hauteur de l'objet détecté.
- On affiche ensuite dans le terminal les valeurs converties en millimètres grâce aux ratios calculés lors de la calibration.
- On affiche également un message à l'écran indiquant à l'utilisateur comment déclencher une mesure.

```

1         cv2.putText(frame, f"Perimetre : {perimetre*ratio_mm_px:.2f} mm", (40, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 1)
2         cv2.putText(frame, f"Aire : {aire*ratio_mm_px**2:.2f} mm2", (40, 65), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 1)
3         cv2.putText(frame, f"Dimensions : {largeur*ratio_mm_px:.2f}x{hauteur*ratio_mm_px:.2f} mm", (40, 90), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 1)
4         cv2.imshow('1', frame)
5
6
7         if key & 0xFF == ord('q'): break

```

- Indépendamment de l'état de calibration, on affiche en permanence sur l'image les dernières valeurs de périmètre, d'aire et de dimensions enregistrées, converties en millimètres.

- On affiche la frame courante dans une fenêtre nommée "1".
- On arrête la boucle si la touche 'q' est pressée.

```
1 cap.release()
2 cv2.destroyAllWindows()
```

- Une fois la boucle terminée, on libère la caméra afin qu'elle reste accessible pour d'autres programmes.
- On ferme toutes les fenêtres ouvertes par OpenCV.

3.3 Traitement.py

3.4 Mesures.py

Ce fichier contient les fonctions qui calculent les différentes mesures de l'objet à partir des contours renvoyés par le traitement de l'image.

- On importe la bibliothèque numpy pour pouvoir travailler sur les contours de type numpy.ndarray.
- On importe la bibliothèque numba pour compiler certains algorithmes trop lourds.

```
1 import numpy as np
2 from numba import njit
```

3.4.1 Calcul du périmètre de l'objet.

```
1 def perimetre(contour):
2     pts = contour.reshape(-1, 2)
3     diff = pts - np.roll(pts, -1, axis=0)
4     distances = np.sqrt((diff**2).sum(axis=1))
5     p = distances.sum()
6     return p
```

- On crée un tableau de points à partir du contour. On utilise la méthode .reshape pour changer la forme du contour de (Nombre de points,1,Coordonnées du point) à (Nombre de points, Coordonnées du point) car OpenCV ajoute une dimension parasite (1) lorsque l'on utilise la fonction cv2.findContours.
- On calcule la différence entre tous les points successifs. Pour cela, on soustrait la liste pts décalée de un point par la méthode np.roll à la liste de pts. On obtient donc une liste de la forme (Nombre de points, $(x_i - x_{i+1}, y_i - y_{i+1})$).
- On calcule ensuite le périmètre en faisant la somme des distances entre les points avec la formule :

$$\sum_{i=0}^{n-1} \sqrt{(x_i - x_{i+1})^2 + (y_i - y_{i+1})^2}$$

La méthode .sum(axis=1) fait la somme des coordonnées $((x_i - x_{i+1})^2, (y_i - y_{i+1})^2)$. Puis np.sqrt calcule la racine de cette somme pour chaque point de diff**2.

- On initie la variable p qui fait la somme de tous les éléments du tableau distances.
- La fonction renvoie la variable p qui correspond au périmètre.

3.4.2 Calcul de l'aire de l'objet.

```
1  def aire(contour):
2      pts = contour.reshape(-1, 2)
3      x = pts[:, 0]
4      y = pts[:, 1]
5      a = 0.5 * abs(np.dot(x, np.roll(y, -1)) - np.dot(y, np.roll(x, -1)))
6      return a
```

— On crée un tableau pts à partir du tableau contour dont la forme a été modifiée avec la méthode .reshape.

3.4.3 Calcul des dimensions de l'objet.

```
1  @njit
2  def dimensions(contour):
3      pts = contour.reshape(-1, 2).astype(np.float64)
4
5      x_min = np.min(pts[:, 0])
6      y_min = np.min(pts[:, 1])
7
8      x_max = np.max(pts[:, 0])
9      y_max = np.max(pts[:, 1])
10
11     aire = (x_max - x_min) * (y_max - y_min)
12     dims = (x_max - x_min, y_max - y_min)
13
14     rad = np.pi / 180
15     c = np.cos(rad)
16     s = np.sin(rad)
17
18     for _ in range(360):
19         for i in range(len(pts)):
20             x, y = pts[i]
21             pts[i][0] = x * c - y * s
22             pts[i][1] = x * s + y * c
23
24             x_min = np.min(pts[:, 0])
25             y_min = np.min(pts[:, 1])
26
27             x_max = np.max(pts[:, 0])
28             y_max = np.max(pts[:, 1])
29             largeur = x_max - x_min
30             hauteur = y_max - y_min
31             aire2 = largeur * hauteur
32
33             if aire2 < aire:
34                 aire = aire2
35                 dims = (largeur, hauteur)
36
37     return dims
```

3.4.4 Fonction mesure(contours).

```
1 def mesures(contours):
2     contour=max(contours, key=perimetre)
3     p = perimetre(contour)
4     a = aire(contour)
5     d = dimensions(contour)
6     return float(p),float(a),float(d[0]),float(d[1])
```

3.5 Optique.py

```
1 import cv2
2 import numpy as np
3 import Traitement as t
4 import mesures as m
5
6 def calibration(contours):
7     flotant = False
8     while flotant is not True :
9         try :
10             longueur_reele_mm = float(input("Longueur reelle de l'
11                                             objet de calibration (mm) : "))
12             flotant = True
13         except :
14             flotant = False
15     if not contours:
16         print("Aucun contour détecté")
17         return -1, -1
18
19     contour = max(contours, key=lambda c: m.aire(c))
20     largeur_px, hauteur_px = m.dimensions(contour)
21     longueur_px = max(largeur_px, hauteur_px)
22
23     ratio_px_par_mm = longueur_px / longueur_reele_mm
24     ratio_mm_par_px = longueur_reele_mm / longueur_px
25
26     print(f"Longueur en pixels : {longueur_px:.1f} px")
27     print(f"Ratio : {ratio_px_par_mm:.3f} px/mm")
28
29     return float(ratio_px_par_mm), float(ratio_mm_par_px)
```